

Teacher:

```
schema_instructions = {
    "json_extraction": (
        'Return JSON with keys: "person_name", "date", "city", "event".'
    ),
    "schema_generation": (
        'Return JSON with keys: "product_name", "price", "in_stock", "tags".'
    ),
    "json_classification": (
        'Return JSON with keys: "label", "rationale". '
        'Label must be one of: positive, negative, neutral.'
    ),
    "json_repair": (
        "Fix the input JSON and return a valid corrected JSON object."
    ),
    "tool_call_arguments": (
        'Return JSON with keys: "origin", "destination", "date", "passengers".'
    ),
}

schema_text = schema_instructions.get(task_type, "Return a valid JSON object.")

return [
    {
        "role": "system",
        "content": (
            "You are a precise data-generation assistant.\n\n"
            "Rules:\n"
            "- Output ONLY valid JSON\n"
            "- No markdown, no code fences\n"
            "- No explanations\n"
            "- Do not include extra keys\n"
            "- Ensure correct types for all fields\n"
            "- Follow the required schema exactly\n"
        ),
    },
    {
        "role": "user",
        "content": (
            f'Task Type: {task_type}\n\n'
        )
    }
]
```

```

        f"{schema_text}\n\n"
        f"Instruction:\n{example['instruction']}\n\n"
        f"Input:\n{example.get('input', '')}\n\n"
        "Return JSON only."
    ),
},
]

```

Student Inference:

```

return (
    "### Instruction:\n"
    f"{instruction}\n\n"
    "### Input:\n"
    f"{input_text}\n\n"
    "### Response:\n"
    f"{output}"
)

```

```

return (
    "### Instruction:\n"
    f"{instruction}\n\n"
    "### Response:\n"
    f"{output}"
)

```

Judge:

```

user_content = (
    "You are evaluating two responses to the same instruction.\n\n"
    "Task:\n"
    "Decide which response is better overall, or declare a tie if they are comparably good.\n\n"
    "Evaluate each response on these dimensions using a 1-5 scale, where 1 is very poor and 5 is excellent:\n"
    "1. Instruction Following\n"
    "2. Correctness\n"
    "3. Clarity\n"
    "4. Completeness\n"
    "5. Hallucination / Fabrication Risk\n\n"
)

```

```

        "Important scoring note:\n"
        "- For hallucination_risk, 1 means very low risk and 5 means very high risk.\n\n"
        "Instruction:\n"
        F"{instruction}\n\n"
    )
    user_content += (
        f"Response A:\n{response_a}\n\n"
        f"Response B:\n{response_b}\n\n"
        "Judge fairly. Do not prefer a response because it appears first, is longer, or sounds more confident.\n"
        "Return valid JSON only with this exact structure:\n"
        "{\n"
        '  "winner": "A" or "B" or "tie",\n'
        '  "reasoning": "1-3 sentence explanation",\n'
        '  "scores": {\n'
        '    "A": {\n'
        '      "instruction_following": 1-5,\n'
        '      "correctness": 1-5,\n'
        '      "clarity": 1-5,\n'
        '      "completeness": 1-5,\n'
        '      "hallucination_risk": 1-5\n'
        '    },\n'
        '    "B": {\n'
        '      "instruction_following": 1-5,\n'
        '      "correctness": 1-5,\n'
        '      "clarity": 1-5,\n'
        '      "completeness": 1-5,\n'
        '      "hallucination_risk": 1-5\n'
        '    }\n'
        '  }\n'
        '}\n\n'
        "Output JSON only."
    )

    return [
        {
            "role": "system",
            "content": "You are a strict, fair evaluator. Return JSON only.",
        },
        {

```

```
"role": "user",  
"content": user_content,  
},  
]
```